Palash Agarwal
PES1UG24CS312

[Github Link]

1A

```
palash@Palash:~/PES1UG24CS312-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
palash@Palash:~/PES1UG24CS312-pes-vcs$
```

1B

```
palash@Palash:~/PES1UG24CS312-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
palash@Palash:~/PES1UG24CS312-pes-vcs$
```

2A

```
palash@Palash:~/PES1UG24CS312-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
palash@Palash:~/PES1UG24CS312-pes-vcs$ git add tree.c
palash@Palash:~/PES1UG24CS312-pes-vcs$ git commit -m "Phase 2: finalize tree construction and pass all tests"
[main 3148b70] Phase 2: finalize tree construction and pass all tests
 1 file changed, 20 insertions(+), 26 deletions(-)
palash@Palash:~/PES1UG24CS312-pes-vcs$
```

2B

```
palash@Palash:~/PES1UG24CS312-pes-vcs$ find .pes/objects -type f
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
palash@Palash:~/PES1UG24CS312-pes-vcs$ xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a         blob 6.hello.
palash@Palash:~/PES1UG24CS312-pes-vcs$
```

3A

```
palash@Palash:~/PES1UG24CS312-pes-vcs$ make pes
make: 'pes' is up to date.
palash@Palash:~/PES1UG24CS312-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
palash@Palash:~/PES1UG24CS312-pes-vcs$ echo "hello" > file1.txt
palash@Palash:~/PES1UG24CS312-pes-vcs$ echo "world" > file2.txt
palash@Palash:~/PES1UG24CS312-pes-vcs$ ./pes add file1.txt file2.txt
palash@Palash:~/PES1UG24CS312-pes-vcs$ ./pes status
Staged changes:
  staged:     a.txt
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.c
  untracked:  README.md
  untracked:  test_objects
  untracked:  pes.h
  untracked:  test_tree.c
  untracked:  index.h
  untracked:  test_sequence.sh
  untracked:  test_tree
  untracked:  tree.c
  untracked:  pes.c
  untracked:  Makefile
  untracked:  test_objects.c
  untracked:  tree.h
  untracked:  commit.c
  untracked:  object.c
  untracked:  commit.h
  untracked:  .gitignore
```

3B

```
palash@Palash:~/PES1UG24CS312-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776534169 6 a.txt
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776534429 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776534438 6 file2.txt
palash@Palash:~/PES1UG24CS312-pes-vcs$
```

4A

```
 1 file changed, 7 insertions(+)
palash@Palash:~/PES1UG24CS312-pes-vcs$ ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Committed: 8ac33325262c... Initial commit
Committed: 496a914a1ce3... Add world
Committed: ea91dfea9f74... Add farewell
commit ea91dfea9f74473b4dccb78935716fa0ae8f432519c29c96ad6c6da3416f4984
Author: Palash Agarwal <PES1UG24CS312>
Date:   1776535240

    Add farewell

commit 496a914a1ce39fa07fc538f7d8e235a2c2c76d270fd71a9ef07e348e7043ffc8
Author: Palash Agarwal <PES1UG24CS312>
Date:   1776535240

    Add world

commit 8ac33325262cc49ba2e328a6c4e97f37cff6737764baf9b2930f577d9a81594e
Author: Palash Agarwal <PES1UG24CS312>
Date:   1776535240

    Initial commit

commit 31a9e31b1a9fb11a6db4381ba175d0fa8a8550f86a32989a323f187a1ac31eba
Author: Palash Agarwal <PES1UG24CS312>
Date:   1776535193

    first5193

fQ

palash@Palash:~/PES1UG24CS312-pes-vcs$
```

4B

```
palash@Palash:~/PES1UG24CS312-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/31/a9e31b1a9fb11a6db4381ba175d0fa8a8550f86a32989a323f187a1ac31eba
.pes/objects/49/6a914a1ce39fa07fc538f7d8e235a2c2c76d270fd71a9ef07e348e7043ffc8
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/8a/c33325262cc49ba2e328a6c4e97f37cff6737764baf9b2930f577d9a81594e
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ea/91dfea9f74473b4dccb78935716fa0ae8f432519c29c96ad6c6da3416f4984
.pes/refs/heads/main
palash@Palash:~/PES1UG24CS312-pes-vcs$
```

4C

```
.pes/refs/heads/main
palash@Palash:~/PES1UG24CS312-pes-vcs$ cat .pes/refs/heads/main
ea91dfea9f74473b4dccb78935716fa0ae8f432519c29c96ad6c6da3416f4984
palash@Palash:~/PES1UG24CS312-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
palash@Palash:~/PES1UG24CS312-pes-vcs$
```

Integration test

```
palash@Palash:~/PES1UG24CS312-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: cecf4827881b... Initial commit

Log after first commit:
commit cecf4827881b5225eb3ec468fa903f1ccfc47e698a814aca5019bdde4f9087a3
Author: Palash Agarwal <PES1UG24CS312>
Date:   1776535468

    Initial commit


--- Second Commit ---
Committed: e7c230e7a4d3... Update file.txt

--- Third Commit ---
Committed: 33cfae8dc49f... Add farewell

--- Full History ---
commit 33cfae8dc49f20c604dc53ab517bb77af3fd2248cbf58e72e8bbc2d2a3f2e483
Author: Palash Agarwal <PES1UG24CS312>
Date:   1776535468
```

```
    Initial commit


--- Second Commit ---
Committed: e7c230e7a4d3... Update file.txt

--- Third Commit ---
Committed: 33cfae8dc49f... Add farewell

--- Full History ---
commit 33cfae8dc49f20c604dc53ab517bb77af3fd2248cbf58e72e8bbc2d2a3f2e483
Author: Palash Agarwal <PES1UG24CS312>
Date:   1776535468

    Add farewell

commit e7c230e7a4d378cff28115e4a0a42e42b8e3e8971d61631ff99194a006190464
Author: Palash Agarwal <PES1UG24CS312>
Date:   1776535468

    Update file.txt

commit cecf4827881b5225eb3ec468fa903f1ccfc47e698a814aca5019bdde4f9087a3
Author: Palash Agarwal <PES1UG24CS312>
Date:   1776535468

    Initial committial commitES1A


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
33cfae8dc49f20c604dc53ab517bb77af3fd2248cbf58e72e8bbc2d2a3f2e483

--- Object Store ---
Objects created:
8
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/33/cfae8dc49f20c604dc53ab517bb77af3fd2248cbf58e72e8bbc2d2a3f2e483
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/ce/cf4827881b5225eb3ec468fa903f1ccfc47e698a814aca5019bdde4f9087a3
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/e7/c230e7a4d378cff28115e4a0a42e42b8e3e8971d61631ff99194a006190464

=== All integration tests completed ===
palash@Palash:~/PES1UG24CS312-pes-vcs$
```

Q5.1: Implementing pes checkout <branch>

A branch in Git (and PES) is stored as a file inside .pes/refs/heads/ that contains the commit hash it points to. To implement pes checkout <branch>, the system must first read the commit hash from the corresponding branch file. Then, it must update the .pes/HEAD file so that it points to the selected branch (e.g., refs/heads/<branch>).

After updating HEAD, the working directory must be updated to reflect the state of the target commit. This involves reading the commit object, retrieving its associated tree, and reconstructing the directory structure. All files in the tree must be written to disk using their corresponding blob objects, and any files that exist in the current working directory but not in the target tree must be removed.

This operation is complex because it involves recursively reconstructing directories, handling file overwrites and deletions, preserving file modes, and ensuring that no uncommitted changes are lost during the process.

Q5.2: Detecting a Dirty Working Directory

To detect uncommitted changes (a dirty working directory), we compare the current working directory with the index. For each file listed in the index, we read the file from disk and compute its hash using the same method used when storing blobs. This computed hash is then compared with the hash stored in the index.

If the hashes differ, the file has been modified and is considered “dirty.” During checkout, we must also compare this file with the version in the target branch. If the working directory version differs from the index, and the target branch version also differs from the index, then a conflict exists. In such a case, checkout must be refused to prevent loss of uncommitted changes.

Q5.3: Detached HEAD

A detached HEAD occurs when the HEAD file points directly to a commit hash instead of a branch reference. In this state, any new commits are created normally, but they are not associated with any branch. As a result, these commits become unreachable from branch references.

If no branch points to these commits, they can eventually be lost or removed during garbage collection. To recover such commits, the user must create a new branch pointing to the commit hash (for example, using a branch command). This makes the commit reachable again and prevents it from being deleted.

Q6.1: Garbage Collection Algorithm

Garbage collection removes objects that are no longer reachable from any branch. The algorithm begins by identifying all root references, which are the commit hashes stored in .pes/refs/heads/. From these roots, the system performs a graph traversal (using depth-first search or breadth-first search) to mark all reachable objects.

During traversal, for each commit, the system visits its associated tree and parent commits. For each tree, it visits all referenced blobs and subtrees. All visited object hashes are stored in a hash set for efficient lookup.

After marking all reachable objects, the system scans the .pes/objects/ directory and deletes any object that is not present in the reachable set.

In a repository with around 100,000 commits and 50 branches, most commits will be reachable, so the algorithm will need to visit roughly 100,000 commits, their corresponding trees, and potentially a large number of blob objects. The total complexity is proportional to the number of reachable objects.

Q6.2: Garbage Collection and Concurrency Issues

Running garbage collection concurrently with a commit operation is dangerous due to race conditions. For example, while a commit is being created, new objects (blobs, trees, or commits) are written to the object store, but the HEAD or branch reference may not yet be updated to point to them. At the same time, garbage collection might scan the object store and determine that these newly created objects are unreachable, since no reference points to them yet, and delete them.

This results in a corrupted repository because the commit operation will later try to reference objects that no longer exist.

Git avoids this problem using several techniques. First, it ensures that objects are written before references are updated, maintaining a safe order of operations. Second, it uses locking mechanisms to prevent concurrent modification of references. Third, it avoids deleting recently created objects by introducing a grace period. Finally, Git maintains reflogs and uses structured garbage collection phases to ensure that objects are only deleted when it is safe to do so.